National University of Computer and Emerging Sciences

Lab Manual 10
Artificial Intelligence

Course Instructor	Ms. Umm-e-Ammarah
Lab Instructor	Muhammad Hasan
Lab Instructor Email ID	m.hasan@lhr.nu.edu.pk
Semester	Spring 2026

FAST - School of Computing
FAST-NU, Lahore Campus

Contents

Lab Manual 10

Objectives:

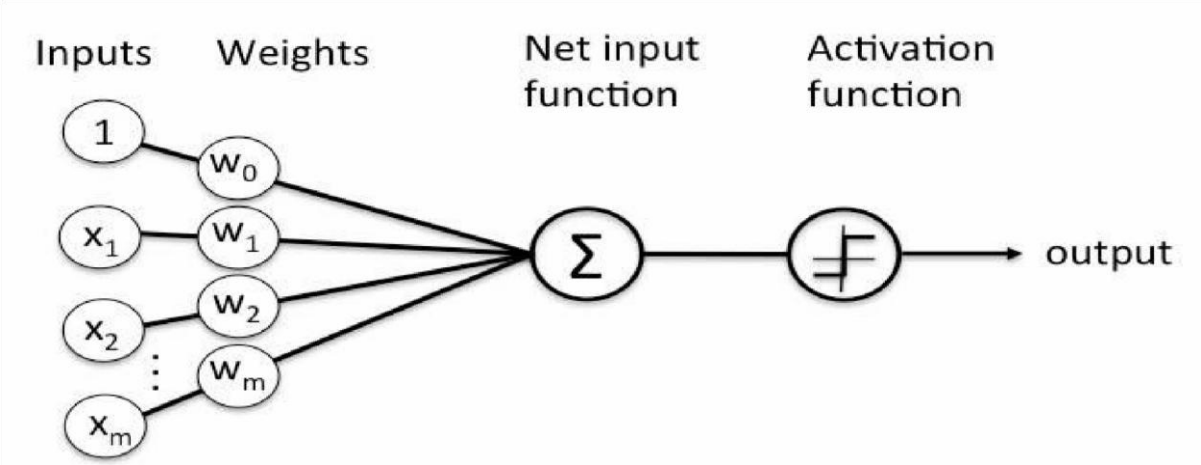
- ✓ Understand single layer perceptron
- ✓ Introduction to TensorFlow and PyTorch
- ✓ Implement perceptron using TensorFlow

Activities:

Activity 1 (Machine Learning Concepts):

1. *Perceptron*

A Perceptron is the simplest form of an artificial neural network used for binary classification. It takes input features, applies weights, adds bias, and passes the result through an activation function.



It computes:

$$f(x) = \sum_{i=1}^{m} (w_i x_i) + b$$

Where:

- w → weights
- x → input features
- b → bias
- m → number of inputs to the Perceptron

Key Concepts

Linear Decision Boundary

- A perceptron can only separate linearly separable data
- Example: AND, OR gates

Activation Function (Step Function)

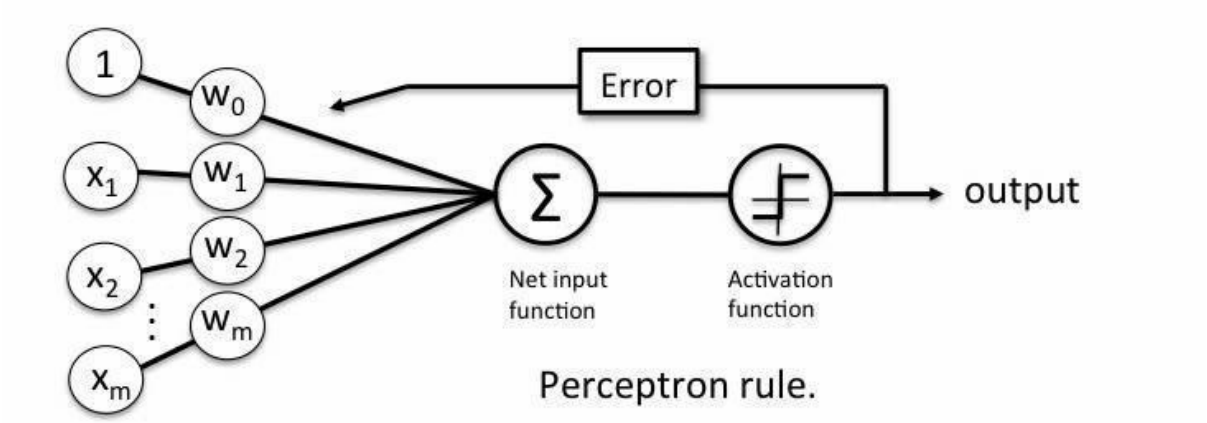
- Converts output into:
  - 1 (Positive class)
  - 0 or -1 (Negative class)

Types of Perceptrons

Type	Description
Single Layer	Works only for linear problems
Multi-Layer	Can solve complex (non-linear) problems

Inputs of a Perceptron

A Perceptron accepts inputs, moderates them with certain weight values, then applies the transformation function to output the final result. The image below shows a Perceptron with a Boolean output.



A Boolean output is based on inputs such as salaried, married, age, past credit profile, etc. It has only two values: Yes and No or True and False. The summation function "∑" multiplies all inputs of "x" by weights "w" and then adds them up as follows:

Activation Functions of Perceptron

The activation function applies a step rule (convert the numerical output into +1 or -1) to check if the output of the weighting function is greater than zero or not.

For example:

if $\sum(w_i x_i) > 0$ then final output = 1 (issue bank loan)

else, final output = 0 or -1 (deny bank loan)

Step function gets triggered above a certain value of the neuron output; else it outputs zero.

Perceptron at a glance

- Weights are multiplied with the input features and decision is made if the neuron is fired or not.
- Activation function applies a step rule to check if the output of the weighting function is greater than zero.
- Linear decision boundary is drawn enabling the distinction between the two linearly separable classes +1 and -1.
- If the sum of the input signals exceeds a certain threshold, it outputs a signal; otherwise, there is no output.
- Types of activation functions include the sign, step, and sigmoid functions.

Logic Gates

Logic gates are the building blocks of a digital system, especially neural networks. In short, they are the electronic circuits that help in addition, choice, negation, and combination to form complex circuits.

Using the logic gates, Neural Networks can learn on their own without you having to manually code the logic. Most logic gates have two inputs and one output.
Each terminal has one of the two binary conditions, low (0) or high (1), represented by different voltage levels. The logic state of a terminal changes based on how the circuit processes data. Based on this logic, logic gates can be categorized into six types:

- AND
- NAND
- OR
- NOR
- NOT
- XOR

2. *Introduction to Deep Learning Frameworks*

What is TensorFlow?

TensorFlow is a deep learning library developed by Google used for building scalable ML models. It allows researchers and developers to build and train neural networks using multi-dimensional arrays called tensors.

Features:

- Tensor-based computation: Core data structure; represents data as n-dimensional arrays.
- Keras API (easy to use): High-level interface in TensorFlow that simplifies model creation, training, and evaluation.
- Supports GPU/TPU: TensorFlow can leverage hardware acceleration for faster training.
- Used in production systems

What is PyTorch?

PyTorch is developed by Meta (Facebook) and is widely used for research. It provides a flexible and dynamic way to define and train neural networks. Unlike TensorFlow's early versions, which used static computation graphs, PyTorch uses dynamic computation graphs, allowing changes in network structure during runtime, very useful for research and experimentation.

Features:

- Dynamic computation graph: Build and modify networks on the fly during execution.
- Automatic Differentiation: The autograd module computes gradients automatically for backpropagation.
- Python-friendly: Seamless integration with Python libraries such as NumPy.
- Strong GPU support: Built-in CUDA support for efficient GPU computations.
- Easy debugging

3. *Perceptron Implementation using PyTorch:*

Step 1: Import necessary libraries

If pytorch is not installed already: `pip install torch torchvision torchaudio`
torch: main PyTorch library
torch.nn: contains neural network layers
torch.optim: contains optimization algorithms

```
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

Step 2: Create and split synthetic dataset

We will create a simple 2-feature synthetic binary-classification dataset for our demonstration and then split it into training and testing.

```
X, y = make_classification(
    n_samples=1000,
    n_features=2,
    n_informative=2,
    n_redundant=0,
    n_repeated=0,
    n_classes=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

Step 3: Standardize the Dataset

Now standardize the dataset to enable faster and more precise computations. Standardization helps the model converge more quickly and often enhances accuracy.

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Step 4: Convert Data to PyTorch Tensors

PyTorch works with a special data structure called a Tensor. A tensor is similar to a NumPy array, but it allows efficient computation and supports automatic differentiation for neural networks. You can think of a tensor as a multi-dimensional array, similar to a NumPy array, but with additional capabilities.

Tensor type	Example	Dimensions
Scalar	5	0D
Vector	[1,2,3]	1D
Matrix	[[1,2],[3,4]]	2D
Higher-dim tensor	image data	3D+

```
import torch

a = torch.tensor(5)                    # scalar
b = torch.tensor([1,2,3])              # vector
c = torch.tensor([[1,2],[3,4]])        # matrix
```

Common Tensor Data Types

The dtype determines how numbers are stored in memory.

dtype	Description
torch.int8	8-bit integer
torch.int16	16-bit integer
torch.int32	32-bit integer
torch.int64	64-bit integer (default for integers)
torch.float16	half precision
torch.float32	standard float (most common)
torch.float64	double precision

```
X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1,1)
# reshaping to convert the target vector into a column vector.

y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1,1)

print(X_train.shape)
print(y_train.shape)
print(X_test.shape)
print(y_test.shape)
```

Step 5: Building a neural network

In PyTorch, neural networks are created by defining a class that inherits from nn.Module. nn.Module is the base class for all neural networks in PyTorch. It provides important functionality such as:

- registering layers
- storing model parameters
- enabling gradient computation
- allowing .train() and .eval() modes
- enabling .parameters() for optimizers

Common PyTorch Layers

Layer	PyTorch Syntax	Purpose	Typical Use
Fully Connected (Dense)	nn.Linear(in_features, out_features)	Connects every input neuron to every output neuron	Standard neural networks
1D Convolution	nn.Conv1d(in_channels, out_channels, kernel_size)	Convolution for sequential data	Time series, NLP
2D Convolution	nn.Conv2d(in_channels, out_channels, kernel_size)	Convolution for image data	Computer vision
3D Convolution	nn.Conv3d(in_channels, out_channels, kernel_size)	Convolution for volumetric data	Medical imaging, video
Max Pooling	nn.MaxPool2d(kernel_size)	Reduces spatial size by taking maximum value	CNN feature extraction
Average Pooling	nn.AvgPool2d(kernel_size)	Reduces spatial size by averaging values	CNN smoothing
Dropout	nn.Dropout(p)	Randomly disables neurons during training	Prevent overfitting
Batch Normalization	nn.BatchNorm1d() / nn.BatchNorm2d()	Normalizes layer outputs	Faster and stable training
Flatten	nn.Flatten()	Converts multidimensional input into vector	Before dense layers
Embedding	nn.Embedding(num_embeddings, embedding_dim)	Converts discrete tokens to dense vectors	NLP models
Recurrent Neural Network	nn.RNN()	Processes sequential data	Text, time series
Long Short-Term Memory	nn.LSTM()	Advanced RNN for long-term dependencies	NLP, speech
Gated Recurrent Unit	nn.GRU()	Simplified LSTM	Sequential models

Common Activation functions

Activation Function	PyTorch Syntax	Output Range	Typical Use
Sigmoid	torch.sigmoid(x)	(0, 1)	Binary classification

ReLU	torch.relu(x)	$[0, \infty)$	Most deep neural networks
Leaky ReLU	nn.LeakyReLU()	$(-\infty, \infty)$ small negative slope	Prevent dead neurons
Tanh	torch.tanh(x)	(-1, 1)	RNNs, older networks
Softmax	torch.softmax(x, dim=1)	Probabilities sum to 1	Multi-class classification
GELU	nn.GELU()	Smooth nonlinear activation	Transformers (BERT, GPT)
ELU	nn.ELU()	Smooth negative values	Deep networks
Softplus	nn.Softplus()	Smooth ReLU-like function	Numerical stability
Identity (No activation)	None	$(-\infty, \infty)$	Regression output

Typical Layer & Activation Combinations

Problem Type	Final Layer	Activation
Binary Classification	nn.Linear(...,1)	Sigmoid
Multi-Class Classification	nn.Linear(...,num_classes)	Softmax
Regression	nn.Linear(...,1)	None
Deep Neural Network Hidden Layer	nn.Linear(...)	ReLU

Our perceptron will contain:

- One linear layer
- One neuron
- Sigmoid activation function

The sigmoid function converts outputs into values between 0 and 1, which is useful for binary classification.

```
class Perceptron(nn.Module):

    def __init__(self):
        super(Perceptron, self).__init__()
        # calls parent class constructor
        self.linear = nn.Linear(2, 1)
        # 2 input features, 1 output
```

```
    def forward(self, x):
        x = self.linear(x)
        x = torch.sigmoid(x)
        return x

model = Perceptron()
```

This creates an instance of the neural network. Internally PyTorch:

- Creates the Linear layer
- Initializes random weights
- Registers parameters for training

```
print(model)
for param in model.parameters():
    print(param)
```

The above class implementation is helpful when you need to control the forward pass. When you just need a sequence of layers you can use nn.Sequential

```
model = nn.Sequential(nn.Linear(2,1),nn.Sigmoid())

print(model)
for param in model.parameters():
    print(param)
```

Step 6: Define Loss Function and Optimizer

To train the neural network we need

- Loss Function: Measures how different the predicted output is from the true output. For binary classification we use Binary Cross Entropy Loss
- Optimizer: Updates the model weights to reduce the loss. We are using the Adam optimizer, which is commonly used in deep learning.

Common Loss Functions in PyTorch

Loss Function	PyTorch Syntax	Used For	Description
Mean Squared Error	nn.MSELoss()	Regression	Measures average squared difference between predicted and true values
Mean Absolute Error	nn.L1Loss()	Regression	Measures average absolute difference between predictions and targets
Binary Cross Entropy	nn.BCELoss()	Binary classification	Compares predicted probabilities with true labels (0 or 1)

Binary Cross Entropy with Logits	nn.BCEWithLogitsLoss()	Binary classification	Combines sigmoid activation and BCE loss in a numerically stable way
Cross Entropy Loss	nn.CrossEntropyLoss()	Multi-class classification	Combines softmax activation and log loss
Negative Log Likelihood	nn.NLLLoss()	Classification	Used with log-softmax outputs
Hinge Loss	nn.HingeEmbeddingLoss()	Margin-based classification	Used in SVM-like models

Common Optimizers in PyTorch

Optimizer	PyTorch Syntax	Key Idea	Typical Use
Stochastic Gradient Descent	optim.SGD()	Updates weights using gradients	Basic optimization
Adam	optim.Adam()	Adaptive learning rate for each parameter	Most commonly used
RMSProp	optim.RMSprop()	Adapts learning rate using moving averages	RNN training
Adagrad	optim.Adagrad()	Adjusts learning rate based on past gradients	Sparse data
AdamW	optim.AdamW()	Adam with better weight decay handling	Modern deep learning models

```
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

Step 7: Train the Model

```
epochs = 50

for epoch in range(epochs):
    # Forward pass
    predictions = model(X_train)
    # Compute loss
    loss = criterion(predictions, y_train)
    # Reset gradients
    optimizer.zero_grad()
    # Backpropagation
    loss.backward()
    # Update weights
    optimizer.step()
```

```
    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}, Loss: {loss.item():.4f}")
```

This code performs the training process of the neural network by repeatedly updating the model's weights so that its predictions become closer to the true labels. The line optimizer.zero_grad() clears any gradients stored from the previous iteration, because PyTorch accumulates gradients by default. Then loss.backward() performs backpropagation, where PyTorch automatically computes the gradients of the loss with respect to all learnable parameters in the model using the chain rule of calculus. These gradients indicate how the weights should change to reduce the loss. After that, optimizer.step() updates the model's weights using the chosen optimization algorithm (Adam in this case), moving them in the direction that reduces the loss. This sequence of (1) forward pass (2) loss computation (3) gradient calculation (4) weight update repeats for every epoch, gradually improving the model's predictions.

Step 8: Model Evaluation

After training we test the model's performance on unseen data.

```
with torch.no_grad():
#Do not track gradients for the operations inside this block.
    y_pred_prob = model(X_test)
    y_pred = (y_pred_prob > 0.5).float()
    accuracy = (y_pred == y_test).sum().item() / y_test.size(0)
print("Accuracy:", accuracy)
```

If you want more detailed metrics such as confusion matrix, F1-score, or classification report use scikit-learn.

```
from sklearn.metrics import confusion_matrix, classification_report

y_pred_np = y_pred.numpy()
y_test_np = y_test.numpy()

print(confusion_matrix(y_test_np, y_pred_np))
print(classification_report(y_test_np, y_pred_np))
```

Activity 2 (Problem Statements / Tasks):

1. *Perceptron for Logic Gate Classification*

You are required to implement a Single Layer Perceptron using PyTorch to model logic gates.

Tasks:

- Create dataset for:
  - AND gate
  - OR gate
- Implement a perceptron using:
  - nn.Linear
  - Sigmoid activation
- Train the model for both datasets

Display:

- Predicted outputs for all inputs

- Accuracy for each gate

Analysis:

- Try applying the same model to XOR gate
- Explain:
  - Why perceptron fails for XOR
  - What type of model is needed instead

2. *Manual Perceptron Implementation*

Implement a perceptron from scratch (without using nn.Module).

Tasks:

- Use:
  - Weight vector w
  - Bias b
- Implement:
  - Forward computation:

$$f(x) = \sum_{i=1}^{m} (w_i x_i) + b$$

  - Step activation function
- Train using a simple dataset (AND gate)

Display:

- Updated weights after each iteration
- Final predictions

Constraint:

- Do NOT use built-in PyTorch layers

3. *Effect of Feature Scaling on Perceptron Training*

You are given a synthetic dataset using make_classification().

Tasks:

- Train perceptron without scaling
- Train perceptron with StandardScaler
- Use same model architecture

Display:

- Training loss per epoch
- Final accuracy (both cases)

Analysis:

- Compare:
- Convergence speed
- Stability of training

Explain:

- Why scaling improves performance

4. *Loss Function Comparison*

Train the same perceptron model using two different loss functions:

- BCELoss
- MSELoss

Tasks:

- Train model using both loss functions separately

- Keep:
  - Same dataset
  - Same epochs
  - Same optimizer

Display:

- Final accuracy
- Loss values

Analysis:

- Which loss performs better for classification?
- Why is BCELoss more suitable?

5. *Model Architecture and Activation Experiment*

Modify the perceptron model and observe behavior.

Tasks:

- Train model using:
  - Sigmoid activation
- Replace activation with:
  - ReLU
- Train both models

Display:

- Accuracy for both models
- Output range of predictions

Analysis:

- Which activation is suitable for binary classification?
- Why does ReLU perform poorly at output layer?